📖 API de Gestión RFID

API **RESTful** para la gestión de datos RFID, desarrollada con **Node.js**, **Express** y **PostgreSQL**.

⚙️ Requisitos

- **Node.js** (v14 o superior)
- **PostgreSQL** (v12 o superior)

🚀 Instalación

1. **Clonar el repositorio**
2. **Instalar dependencias**:

   npm install

3. **Configurar variables de entorno**:
   Copiar el archivo .env.example a .env y editar los valores necesarios.
4. **Configurar la aplicación**:
   Ajustar los parámetros en config/app.json.
5. **Iniciar el servidor**:

   npm start

👉 Para entorno de desarrollo:

npm run dev

🏗️ Estructura de la API

🔑 Autenticación

Todas las rutas requieren una **API Key válida** que debe enviarse en el encabezado:

x-api-key: TU_API_KEY

📍 Endpoints

🔷 EPC_readings

- GET /api/epc-readings/stock → Obtiene registros con Stock = 1 y Used != 1

    - Parámetros opcionales: devid, startDate, endDate
- GET /api/epc-readings/stock-temporal → Obtiene registros con Stock = 0 y Used != 1
    - Parámetros opcionales: devid, startDate, endDate
- DELETE /api/epc-readings/stock-temporal → Elimina todos los registros con Stock = 0

◈ **Bin**

- GET /api/bin/stock-consumption → Obtiene registros de consumo
    - Parámetros opcionales: devid, patient, operating_room, startDate, endDate
- DELETE /api/bin → Elimina todos los registros de la tabla **Bin**

◈ **User_readings**

- GET /api/user-readings/user-door → Obtiene registros de usuarios en puertas
    - Parámetros opcionales: devid, startDate, endDate
- DELETE /api/user-readings → Elimina todos los registros de la tabla **User_readings**

◈ **user_room**

- GET /api/user-room → Obtiene todos los registros
- GET /api/user-room/epc/:epc → Obtiene registros por EPC
- GET /api/user-room/door/:door → Obtiene registros por puerta
- POST /api/user-room → Inserta un nuevo registro
    - Cuerpo:

      { "epc": "string", "door": "string", "state": number }
- PUT /api/user-room/:id → Actualiza un registro existente
- DELETE /api/user-room/:id → Elimina un registro existente
- DELETE /api/user-room → Elimina todos los registros

◈ **EPC_master**

- GET /api/epc-master → Obtiene todos los registros
- GET /api/epc-master/epc/:epc → Obtiene registros por EPC

- GET /api/epc-master/description/:description → Obtiene registros por descripción (búsqueda parcial)
- GET /api/epc-master/almacen/:almacen → Obtiene registros por almacén
- POST /api/epc-master → Inserta un nuevo registro
- PUT /api/epc-master/:id → Actualiza un registro existente
- DELETE /api/epc-master/:id → Elimina un registro existente

⚙ Configuración

El archivo config/app.json define los parámetros principales:

- **Servidor**
  - port: Puerto del servidor
  - apiKey: Clave de API para autenticación
- **Base de Datos**
  - host: Host de la base de datos
  - port: Puerto de la base de datos
  - name: Nombre de la base de datos
  - user: Usuario de la base de datos
  - password: Contraseña
  - ssl: Activar SSL si es necesario
- **Logs**
  - level: Nivel (debug, info, warn, error)
  - maxFiles: Número máximo de archivos de log
  - maxSize: Tamaño máximo de cada archivo

🌐 Interfaz Web

Disponible en la ruta raíz /, permite:

- Explorar endpoints disponibles
- Filtrar resultados
- Realizar operaciones **CRUD** en user_room y EPC_master
- Eliminar registros de forma masiva
- Visualizar datos en tablas

📝 Logs

Los registros se almacenan en la carpeta logs:

- error.log → Solo errores
- combined.log → Todos los logs

⚠ Manejo de Errores

El sistema incluye:

1. Registro automático de errores en logs
2. Respuestas de error estructuradas
3. Detalles adicionales en modo desarrollo

📤 Estructura de Respuestas

✅ Éxito

```
{
  "data": [...],
  "count": number
}
```

❌ Error

```
{
  "error": {
    "message": "string",
    "status": number,
    "timestamp": "ISO string"
  }
}
```

🔒 Seguridad

- Todas las rutas requieren **API Key**
- Autenticación mediante encabezado x-api-key
- Protección con **CORS** y **Helmet**
- Validación de entrada en todos los endpoints
